

MINIMUM SPANNING TREE ALGORITHMS

Graph Theory — Week 2 Group Assignment



Ahmad Farras Favian Al Efasi - 5025251005

Daniel Pedrosaputra - 5025251171

//

Institut Teknologi Sepuluh Nopember

2026

1. ALGORITHM INTRODUCTION

a. Prim's Algorithm

Prim's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a weighted, connected, undirected graph. It starts from a selected vertex and gradually grows the spanning tree by choosing the minimum-weight edge that connects a vertex inside the tree to a vertex outside the tree. It maintains a set of vertices already included in the MST and considers the edges connecting them to unvisited vertices. A Min-Priority Queue can be used to efficiently select the edge with the smallest weight.

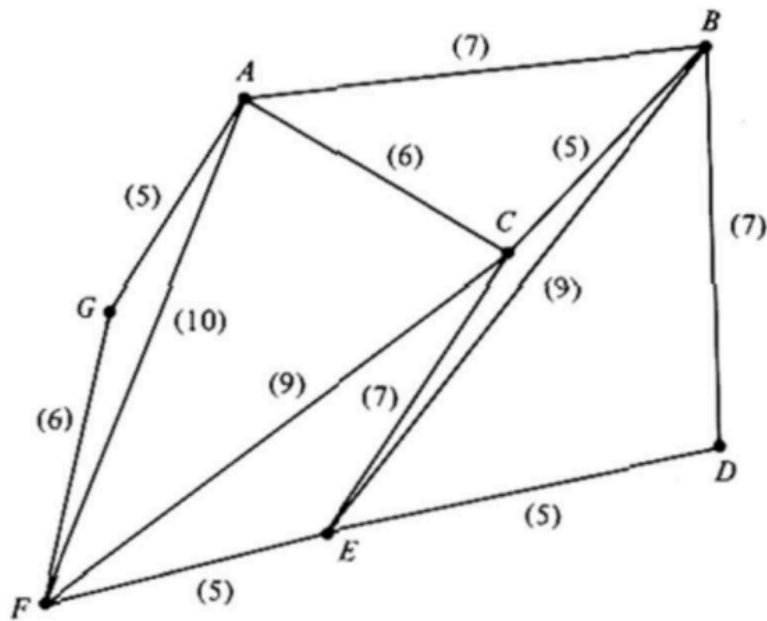
b. Kruskal's Algorithm

Kruskal's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree of a weighted graph. Instead of starting from a particular vertex, it considers all edges in increasing order of their weights.

The algorithm adds an edge to the MST if adding it does not create a cycle. It continues until all vertices are connected. A Disjoint Set Union (DSU) or Union-Find data structure can be used to efficiently detect whether adding an edge would create a cycle.

c. Borůvka's Algorithm

Borůvka's Algorithm is another greedy algorithm for finding a Minimum Spanning Tree. It initially treats every vertex as a separate component. In each phase, every component selects its cheapest outgoing edge. The selected edges are then added to the MST, merging multiple components together. This process is repeated until only one connected component remains. Unlike Prim's and Kruskal's algorithms, Borůvka's Algorithm works by simultaneously connecting multiple components in each phase.



2. ALGORITHM FAILURE ANALYSIS

a. No Failure

The original graph with no failure consists of seven vertices: A, B, C, D, E, F, and G connected by weighted undirected edges. The objective is to select a set of edges that connects all vertices without forming a cycle while minimizing the total edge weight.

Why Prim's?

Prim's algorithm is suitable because it grows one connected MST from a starting vertex. Starting from A, the algorithm can repeatedly choose the cheapest edge connecting the current tree to a vertex outside the tree. This makes the construction of the MST easy to follow step by step.

Why Kruskal?

Kruskal's algorithm is suitable because the graph has a relatively small number of edges and the weights can be sorted directly. The algorithm can select the globally cheapest edges first while using cycle detection to avoid invalid additions.

Why Borůvka?

Borůvka's algorithm is used as the additional MST algorithm required by the assignment. It provides a different strategy because every current component selects its cheapest outgoing edge in each phase, allowing several components to merge simultaneously.

b. Edge Failure

For the edge-failure simulation, edge A–G with weight 5 is assumed to be unavailable. The algorithms must ignore this edge and construct an MST using the remaining edges. Because A–G is one of the edges in the original MST, the total MST cost increases after the failure.

Expected result: the replacement MST has a total cost 34. One valid replacement set is C–B (5), F–E (5), E–D (5), G–F (6), A–C (6), and C–E (7).

c. Node Failure

For the node-failure simulation, vertex G is assumed to be unavailable. Vertex G and all edges incident to it are removed from the graph. The algorithms then construct an MST for the remaining six vertices.

Expected result: the remaining graph can be connected with an MST of total cost 28 using C–B (5), F–E (5), E–D (5), A–C (6), and C–E (7).

3. ALGORITHM SIMULATION

a. Prim's Algorithm

Starting vertex: A

Step	Selected Edge	Weight	Vertices in MST
1	A–G	5	A, G
2	G–F	6	A, G, F
3	F–E	5	A, G, F, E
4	E–D	5	A, G, F, E, D
5	A–C	6	A, G, F, E, D, C
6	C–B	5	A, G, F, E, D, C, B

Final MST: A–G (5) → G–F (6) → F–E (5) → E–D (5) → A–C (6) → C–B (5).

Total cost = 32.

b. Kruskal's Algorithm

Edges are processed from the smallest weight to the largest. An edge is accepted only when it does not form a cycle.

Order	Edge	Weight	Decision
1	A–G	5	Accept
2	C–B	5	Accept
3	F–E	5	Accept
4	E–D	5	Accept
5	G–F	6	Accept
6	A–C	6	Accept

The remaining edges are not needed because all seven vertices are already connected by six accepted edges.

Final MST cost = 32.

c. Borůvka's Algorithm

Phase 1: each initial component selects its cheapest outgoing edge. A valid selection is A–G (5), B–C (5), E–D (5), and E–F (5). These edges produce three components: [A,G], [B,C], and [D,E,F].

Phase 2: the component [A,G] selects G–F (6), while [B,C] selects A–C (6). These edges merge the remaining components into one connected component.

Phase	Component	Cheapest Outgoing Edge	Weight	Decision
1	A	A–G	5	Add
	B	B–C	5	Add
	C	C–B	5	Already selected
	D	D–E	5	Add
	E	E–F	5	Add
	F	F–E	5	Already selected
	G	G–A	5	Already selected
After Phase 1	[A,G], [B,C], [D,E,F]			3 components
2	[A,G]	G–F	6	Add
	[B,C]	C–A	6	Add
	[D,E,F]	F–G	6	Already selected
After Phase 2	[A,B,C,D,E,F,G]			Completed

Final MST cost = 5 + 5 + 5 + 5 + 6 + 6 = 32.

4. FAILURE SIMULATION

a. Edge Failure: A–G (5)

When A–G is unavailable, Prim's, Kruskal's, and Borůvka's algorithms must exclude the failed edge. The next cheapest valid connection for the affected structure is used instead. A valid replacement MST is C–B (5), F–E (5), E–D (5), G–F (6), A–C (6), and C–E (7), giving a total cost of 34.

The main adaptation is therefore to treat the failed edge as removed from the graph and continue the MST construction using the remaining valid edges.

b. Node Failure: G

When vertex G fails, G and its incident edges A–G, G–F, and A–F are removed. The algorithms reconstruct the MST over the remaining vertices A, B, C, D, E, and F.

A valid MST is C–B (5), F–E (5), E–D (5), A–C (6), and C–E (7), with a total cost of 28.

The main adaptation is to remove the failed node and all edges connected to it before running the MST algorithm again.

5. COMPARISON

Problem	Algorithm	MST Edges	Total Cost	Result
Original	Prim's	6	32	Valid MST
	Kruskal	6	32	Valid MST
	Borůvka	6	32	Valid MST
Edge A–G failed	Prim's	6	34	Reconstructed MST
	Kruskal	6	34	Reconstructed MST
	Borůvka	6	34	Reconstructed MST
Node G failed	Prim's	5	28	Reconstructed MST
	Kruskal	5	28	Reconstructed MST
	Borůvka	5	28	Reconstructed MST

Note: the failure rows show the expected MST cost after the same graph modification. The exact edge-selection order may differ between algorithms while still producing a valid MST with the same minimum total cost.

6. CONCLUSION

Through the implementation and simulation of Prim's, Kruskal's, and Borůvka's algorithms, we demonstrated three different greedy strategies for constructing a Minimum Spanning Tree. Prim's grows one tree from a starting vertex, Kruskal's selects globally light edges while avoiding cycles, and Borůvka's simultaneously connects multiple components through their cheapest outgoing edges.

For the original graph, all three algorithms can produce an MST with a total cost of 32. This shows that different MST strategies can reach the same optimal result even though their selection processes are different.

The failure simulations also show that an MST algorithm must adapt to changes in the graph. When edge A–G fails, the algorithm removes that edge and finds a replacement connection, increasing the minimum total cost to 34. When node G fails, G and its incident edges are removed and the remaining graph has an MST with total cost 28.

Overall, the choice of MST algorithm depends on the graph structure and implementation requirements, while the correctness of the result depends on maintaining connectivity and avoiding cycles.

Repository: https://github.com/R3n254/Group_2_Graph-Theory_MST

Prompt:

1. Explain to me what I must do for the assignment [Refurbished W2 Tree and Spanning.pdf].
2. Is Boruvka's Algorithm usable in this case?